Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

Last modified 2025-11-12.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2024 semester.

Previous Chapter | Table of contents | Next Chapter

Launch in Deepnote

Chapter 9
Object Detection and Segmentation

Our study of geometric perception gave us good tools for estimating the pose of a known object. These algorithms can produce highly accurate estimates, but are still subject to local minima. When the scenes get more cluttered/complicated, or if we are dealing with many different object types, they really don't offer an adequate solution by themselves.

Deep learning has given us data-driven solutions that complement our geometric approaches beautifully. Finding correlations in massive datasets has proven to be a fantastic way to provide practical solutions to these more "global" problems like detecting whether the mustard bottle is even in the scene, segmenting out the portion of the image / point cloud that is relevant to the object, and even in providing a rough estimate of the pose that could be refined with a geometric method.

There are many sources of information about deep learning on the internet, and I have no ambition of replicating nor replacing them here. But this chapter does being our exploration of deep perception in manipulation, and I feel that I need to give just a little context.

9.1 Getting to big data

9.1.1 Crowd-sourced annotation datasets

The modern revolution in computer vision was unquestionably fueled by the availability of massive annotated datasets. The most famous of all is ImageNet, which eclipsed previous datasets with the number of images and the accuracy and usefulness of the labels[1]. Fei-fei Li, who led the creation of ImageNet has been giving talks that give some nice historical perspective on how ImageNet came to be. Here is one (slightly) tailored to robotics and even manipulation; you might start here.

[1] describes the annotations available in ImageNet:

> ... annotations fall into one of two categories: (1) image-level annotation of a binary label for the presence or absence of an object class in the image, e.g., "there are cars in this image" but "there are no tigers," and (2) object-level annotation of a tight bounding box and class label around an object instance in the image, e.g., "there is a screwdriver centered at position (20,25) with width of 50 pixels and height of 30 pixels".

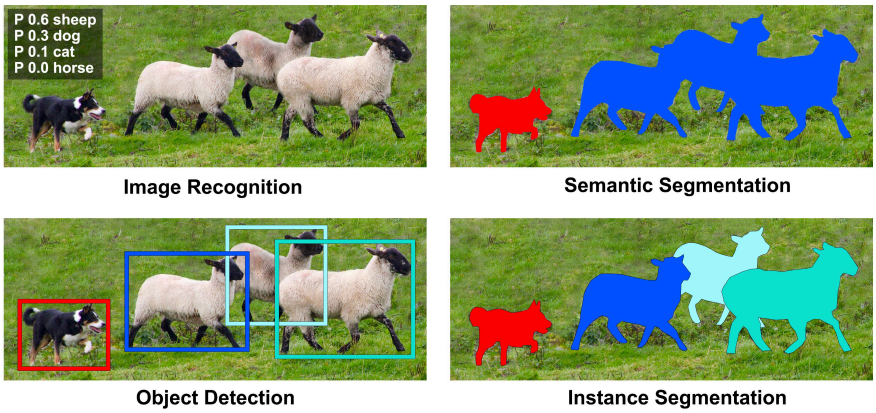


Figure 9.1 - A sample annotated image from the COCO dataset, illustrating the difference between image-level annotations, object-level annotations, and segmentations at the class/semantic- or instance- level..

In practice, ImageNet enabled object detection. The COCO dataset similarly enabled pixel-wise *instance-level* segmentation [2], where distinct instances of a class are given a unique label (and also associated with the class label). COCO has fewer object categories than ImageNet, but more instances per category. It's still shocking to me that they were able to get 2.5 million images labeled at the pixel level. I remember some of the early projects at MIT when crowd-sourced image labeling was just beginning (projects like LabelMe [3]); Antonio Torralba used to joke about how surprised he was about the accuracy of the (nearly) pixel-wise annotations that he was able to crowd-source (and that his mother was a particularly prolific and accurate labeler)!

Instance segmentation turns out to be an very good match for the perception needs we have in manipulation. In the last chapter we found ourselves with a bin full of YCB objects. If we want to pick out only the mustard bottles, and pick them out one at a time, then we can use a deep network to perform an initial instance-level segmentation of the scene, and then use our grasping strategies on only the segmented point cloud. Or if we do need to estimate the pose of an object (e.g. in order to place it in a desired pose), then segmenting the point cloud can also dramatically improve the chances of success with our geometric pose estimation algorithms.

9.1.2 Segmenting new classes via fine tuning

The ImageNet and COCO datasets contain labels for a variety of interesting classes, including cows, elephants, bears, zebras and giraffes. They have a few classes that are more relevant to manipulation (e.g., plates, forks, knives, and spoons), but they don't have a mustard bottle nor a can of potted meat like we have in the YCB dataset. So what are we do to? Must we produce the same image annotation tools and pay for people to label thousands of images for us?

One of the most amazing and magical properties of the deep architectures that have been working so well for instance-level segmentation is their ability to transfer to new tasks ("transfer learning"). A network that was pre-trained on a large dataset like ImageNet or COCO can be fine-tuned with a relatively much smaller amount of labeled data to a new set of classes that are relevant to our particular application. In fact, the architectures are often referred to as having a "backbone" and a "head" -- in order to train a new set of classes, it is often possible to just pop off the existing head and replace it with a new head for the new labels. A relatively small amount of training with a relatively small dataset can still achieve surprisingly robust performance. Moreover, it seems that training initially on the diverse dataset (ImageNet or COCO) is actually important to learn the robust perceptual representations that work for a broad class of perception tasks. Incredible!

This is great news! But we still need some amount of labeled data for our objects of interest. The last few years have seen a number of start-ups based purely on the business model of helping you get your dataset labeled. But thankfully, this isn't our only option.

9.1.3 Annotation tools for manipulation

Just as projects like LabelMe helped to streamline the process of providing pixel-wise annotations for images downloaded from the web, there are a number of tools that have been developed to streamline the annotation process for robotics. One of the earliest examples was LabelFusion, which combines geometric perception of point clouds with a simple user interface to very rapidly label a large number of images[4].

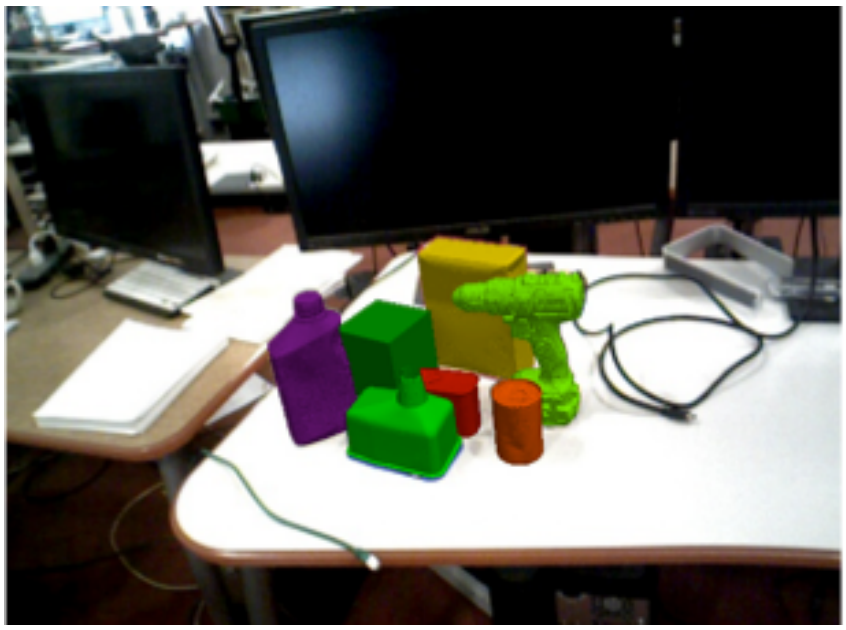


Figure 9.2 - A multi-object scene from LabelFusion [4]. (Mouse over for animation)

In LabelFusion, the user provides multiple RGB-D images of a static scene containing some objects of interest, and the CAD models for those objects. LabelFusion uses a dense reconstruction algorithm, ElasticFusion[5], to merge the point clouds from the individual images into a single dense reconstruction; this is just another instance of the point cloud registration problem. The dense reconstruction algorithm also localizes the camera relative to the point cloud. To localize a particlar object, like the drill in the image above, LabelFusion provides a simple gui that asks the user to click on three points on the model and three points in the scene to establish the "global" correspondence, and then runs ICP to refine the pose estimate. In addition to this one registration providing labeled poses in *all* of the original images, the pixels from the CAD model can be "rendered" on top of all of the images in the established pose giving beautiful pixel-wise labels.

Tools like LabelFusion can be use to label large numbers of images very quickly (three clicks from a user produces ground truth labels in many images).

9.1.4 Synthetic datasets

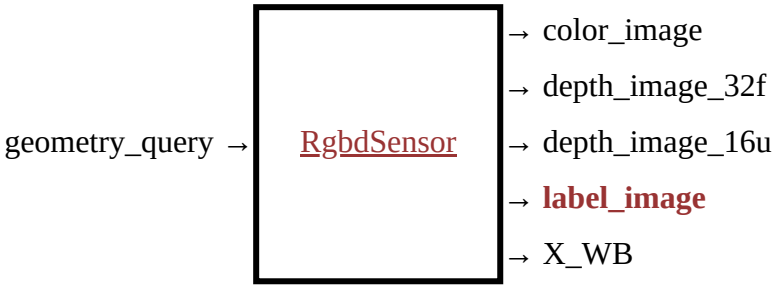
All of this real world data is incredibly valuable. But we have another super powerful tool at our disposal: simulation! Computer vision researchers have traditionally been very skeptical of training perception systems on synthetic images, but as game-engine quality physics-based rendering has become a commodity technology, roboticists have been using it aggressively to supplement or even replace their real-world datasets. The annual robotics conferences now feature regular workshops and/or debates on the topic of "sim2real". For any specific scene or narrow class of objects, we can typically generate accurate enough art assets (with material properties that are often still tuned by an artist) and environment maps / lighting conditions that rendered images can be highly effective in a training dataset. The bigger question is whether we can generate a *diverse* enough set of data with distributions representative of the real world to train robust feature detectors in the way that we've managed to train with ImageNet. But for many serious robotics groups, synthetic data generation pipelines have significantly augmented or even replaced real-world labeled data.

There is a subtle reason for this. Human annotations on real data, although they can be quite good, are never perfect. Labeling errors can put a ceiling on the total performance achievable by the learning system[6]. Even if we admit the gap between rendered images and natural images, at some point the ability to generate arbitrarily large datasets with perfect pixel-wise labels actually enables

training on synthetic datasets to surpass the performance for training on real data even when evaluated on real-world test sets.

For the purposes of this chapter, I aim to train an instance-level segmentation system that will work well on our simulated images. For this use case, there is (almost) no debate! Leveraging the pre-trained backbone from COCO, I will use only synthetic data for fine tuning.

You may have noticed it already, but the `RgbdSensor` that we've been using in Drake actually has a "label image" output port that we haven't used yet.



This output port exists precisely to support the perception training use case we have here. It outputs an image that is identical to the RGB image, except that every pixel is "colored" with a unique instance-level identifier.

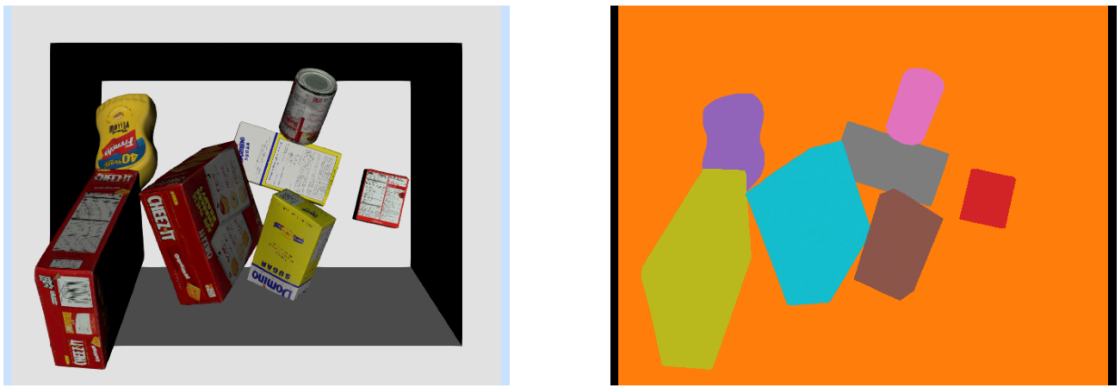


Figure 9.3 - Pixelwise instance segmentation labels provided by the "label image" output port from `RgbdSensor`. I've remapped the colors to be more visually distinct.

Example 9.1 (Generating training data for instance segmentation)

I've provided a simple script that runs our "clutter generator" from our bin picking example that drops random YCB objects into the bin. After a short simulation, I render the RGB image and the label image, and save them (along with some metadata with the instance and class identifiers) to disk.

I've verified that this code *can* run on Colab, but to make a dataset of 10k images using this un-optimized process takes about an hour on my big development desktop. And curating the files is just easier if you run it locally. So I've provided this one as a python script instead.

```
python3 segmentation/segmentation_data.py
```

You can also feel free to skip this step! I've uploaded the 10k images that I generated here. We'll download that directly in our training notebook.

9.1.5 Self-supervised learning

9.1.6 Even bigger datasets

With the rise of large language models (LLMs) came a very natural question: how do we obtain a "foundation model" for computer vision? This would be loosely defined as a model that had impressive zero-shot prediction performance on basically any new image, without prompting and a small number of interactions with a non-expert user replacing the need for fine-tuning on a domain-specific dataset.

Segment Anything [7] came out earlier in 2023; it is a foundation model for segmentation tasks. The associated dataset, SA-1B, is dramatically larger that pre-existing datasets like COCO in terms of the number of images,the resolution of the images, and the number of labeled segmentations. It's enormous scale was enabled by a "data-engine" which used increasingly powerful versions of the Segment Anything models to provide initial segmentation labels; this output was then passed to expert paid image labelers who could adjust/correct the labels and add labels for increasingly obscure parts of the image that the model had missed. Perhaps fine-tuning on our robot-specific datasets is, or will soon be, a thing of the past.

9.2 Object detection and segmentation

There is a lot to know about modern object detection and segmentation pipelines. I'll stick to the very basics.

For image recognition (see Figure 1), one can imagine training a standard convolutional network that takes the entire image as an input, and outputs a probability of the image containing a sheep, a dog, etc. In fact, these architectures can even work well for semantic segmentation, where the input is an image and the output is another image; a famous architecture for this is the Fully Convolutional Network (FCN) [8]. But for object detection and instance segmentation, even the number of outputs of the network can change. How do we train a network to output a variable number of detections?

The mainstream approach to this is to first break the input image up into many (let's say on the order of 1000) overlapping regions that might represent interesting sub-images. Then we can run our favorite image recognition and/or segmentation network on each subimage individually, and output a detection for each region that that is scored as having a high probability. In order to output a tight bounding box, the detection networks are also trained to output a "bounding box refinement" that selects a subset of the final region for the bounding box. Originally, these region proposals were done with more traditional image preprocessing algorithms, as in R-CNN (Regions with CNN Features)[9]. But the "Fast" and "Faster" versions of R-CNN replaced even these preprocessing with learned "region proposal networks"[10, 11].

For instance segmentation, we will use the very popular Mask R-CNN network which puts all of these ideas, using region proposal networks and a fully convolutional networks for the object detection and for the masks [12]. In Mask R-CNN, the masks are evaluated in parallel from the object detections, and only the masks corresponding to the most likely detections are actually returned. At the time of this writing, the latest and most performant implementation of Mask R-CNN is available in the Detectron2 project from Facebook AI Research. But that version is not quite as user-friendly and clean as the original version that was released in the PyTorch `torchvision` package; we'll stick to the `torchvision` version for our experiments here.

Example 9.2 (Fine-tuning Mask R-CNN for bin picking)

The following notebook loads our 10k image dataset and a Mask R-CNN network pre-trained on the COCO dataset. It then replaces the head of the pre-trained network with a new head with the right number of outputs for our YCB recognition task, and then runs just a 10 epochs of training with my new dataset.

Open in Colab (Training Notebook)

Training a network this big (it will take about 150MB on disk) is not fast. I strongly recommend hitting play on the cell immediately after the training cell while you are watching it train so that

the weights are saved and downloaded even if your Colab session closes. But when you're done, you should have a shiny new network for instance segmentation of the YCB objects in the bin!

I've provided a second notebook that you can use to load and evaluate the trained model. If you don't want to wait for your own to train, you can examine the one that I've trained!

Open in Colab (Inference Notebook)

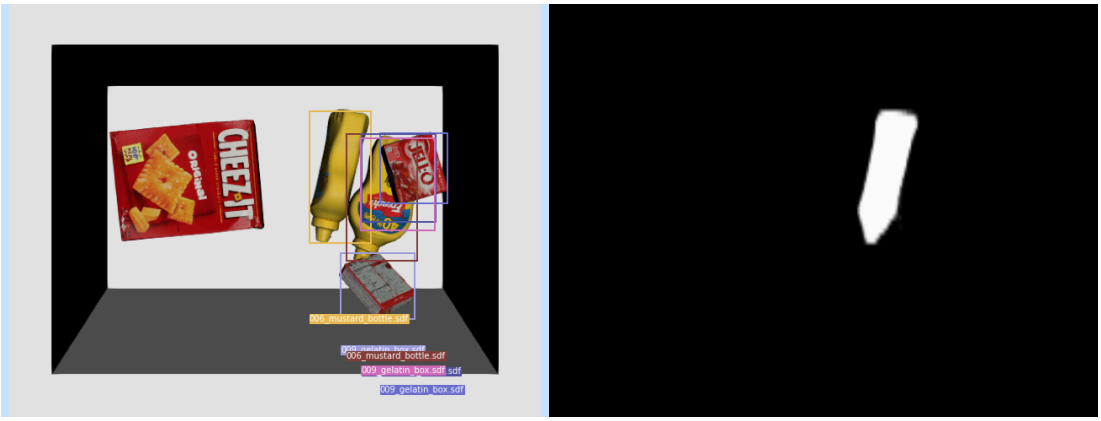


Figure 9.4 - Outputs from the Mask R-CNN inference. (Left) Object detections. (Right) One of the instance masks.

9.3 Putting it all together

We can use our Mask R-CNN inference in a manipulation to do selective picking from the bin...

9.4 Variations and Extensions

9.4.1 Pretraining wth self-supervised learning

9.4.2 Leveraging large-scale models

One of the goals for these notes is to consider "open-world" manipulation -- making a manipulation pipeline that can perform useful tasks in previously unseen environments and with unseen models. How can we possibly provide labeled instances of every object the robot will ever have to manipulate?

The most dramatic examples of open-world reasoning have been coming from the so-called "foundation models"[13]. The foundation model that has been adopted most quickly into robotics research is the large vision + text model, CLIP [14].

More coming soon...

9.5 Exercises

Exercise 9.1 (Label Generation)

For this exericse, you will look into a simple trick to automatically generate training data for Mask-RCNN. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Automatically generate mask labels from pre-processed point clouds.
b. Analyze the applicability of the method for more complex scenes.
c. Apply data augmentation techniques to generate more training data.

Exercise 9.2 (Segmentation + Antipodal Grasping)

For this exercise, you will use Mask-RCNN and our previously developed antipodal grasp strategy to select a grasp given a point cloud. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Automatically filter the point cloud for points that correspond to our intended grasped object
b. Analyze the impact of a multi-camera setup.
c. Consider why filtering the point clouds is a useful step in this grasping pipeline.
d. Discuss how we could improve this grasping pipeline.

Exercise 9.3 (Vision-Language Segmentation)

For this exercise, you will explore how Vision-Language Models (VLMs) and the Segment Anything Model (SAM) can be combined to achieve language-driven object segmentation. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Analyze SAM's segmentation capabilities and understand its limitations in object identification
b. Use a Vision-Language Model to generate bounding boxes from natural language prompts
c. Combine VLM-generated bounding boxes with SAM to produce precise segmentation masks for specified objects

References

1. Olga Russakovsky and Jia Deng and Hao Su and Jonathan Krause and Sanjeev Satheesh and Sean Ma and Zhiheng Huang and Andrej Karpathy and Aditya Khosla and Michael Bernstein and others, "Imagenet large scale visual recognition challenge", *International journal of computer vision*, vol. 115, no. 3, pp. 211--252, 2015.

2. Tsung-Yi Lin and Michael Maire and Serge Belongie and James Hays and Pietro Perona and Deva Ramanan and Piotr Dollár and C Lawrence Zitnick, "Microsoft coco: Common objects in context", *European conference on computer vision* , pp. 740--755, 2014.

3. Bryan C Russell and Antonio Torralba and Kevin P Murphy and William T Freeman, "LabelMe: a database and web-based tool for image annotation", *International journal of computer vision*, vol. 77, no. 1-3, pp. 157--173, 2008.

4. Pat Marion and Peter R. Florence and Lucas Manuelli and Russ Tedrake, "A Pipeline for Generating Ground Truth Labels for Real RGBD Data of Cluttered Scenes", *International Conference on Robotics and Automation (ICRA), Brisbane, Australia*, May, 2018. [link]

5. Thomas Whelan and Renato F Salas-Moreno and Ben Glocker and Andrew J Davison and Stefan Leutenegger, "ElasticFusion: Real-time dense SLAM and light source estimation", *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697--1716, 2016.

6. Curtis G Northcutt and Anish Athalye and Jonas Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks", *arXiv preprint arXiv:2103.14749*, 2021.

7. Alexander Kirillov and Eric Mintun and Nikhila Ravi and Hanzi Mao and Chloe Rolland and Laura Gustafson and Tete Xiao and Spencer Whitehead and Alexander C Berg and Wan-Yen Lo and others, "Segment anything", *arXiv preprint arXiv:2304.02643*, 2023.

8. Jonathan Long and Evan Shelhamer and Trevor Darrell, "Fully convolutional networks for semantic segmentation", *Proceedings of the IEEE conference on computer vision and pattern recognition* , pp. 3431--3440, 2015.

9. Ross Girshick and Jeff Donahue and Trevor Darrell and Jitendra Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation", *Proceedings of the IEEE conference on computer vision and pattern recognition* , pp. 580--587, 2014.

10. Ross Girshick, "Fast r-cnn", *Proceedings of the IEEE international conference on computer vision* , pp. 1440--1448, 2015.

11. Shaoqing Ren and Kaiming He and Ross Girshick and Jian Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks", *Advances in neural information processing systems* , pp. 91--99, 2015.

12. Kaiming He and Georgia Gkioxari and Piotr Dollár and Ross Girshick, "Mask R-CNN", *Proceedings of the IEEE international conference on computer vision* , pp. 2961--2969, 2017.

13. Rishi Bommasani and Drew A Hudson and Ehsan Adeli and Russ Altman and Simran Arora and Sydney von Arx and Michael S Bernstein and Jeannette Bohg and Antoine Bosselut and Emma Brunskill and others, "On the opportunities and risks of foundation models", *arXiv preprint arXiv:2108.07258*, 2021.

14. Alec Radford and Jong Wook Kim and Chris Hallacy and Aditya Ramesh and Gabriel Goh and Sandhini Agarwal and Girish Sastry and Amanda Askell and Pamela Mishkin and Jack Clark and others, "Learning transferable visual models from natural language supervision", *International Conference on Machine Learning* , pp. 8748--8763, 2021.

Previous Chapter Table of contents Next Chapter

Accessibility